

SOFTWARE ENGINEERING – LAB 1

Name: Nidhi S

SRN: PES1UG24AM919

CLASS: AIML Section G

TOPIC NO.: 38

Title: **Dropshipping Inventory & Order Orchestrator**

Requirements Table

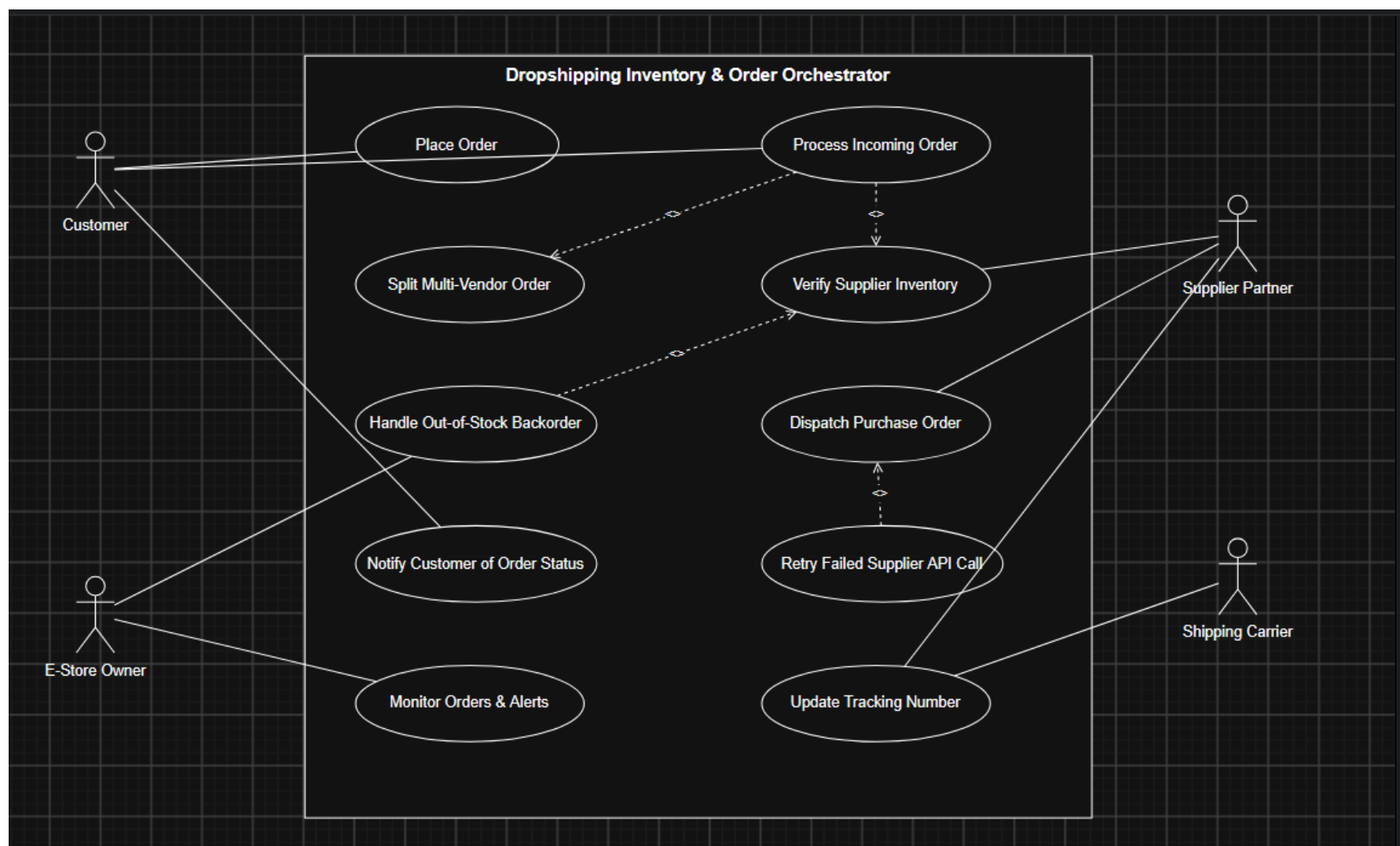
Stakeholders / Actors: E-Store Owner, Supplier Partner, Customer, Shipping Carrier

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse inbound Shopify/WooCommerce order webhooks, verify supplier API inventory, and auto-dispatch supplier purchase orders.	High	Pass: Supplier PO triggered with customer delivery address. Fail: Order placed for out-of-stock supplier SKU.	Automates the core order-to-fulfillment pipeline and removes manual order entry, which is the main source of dispatch delay.
FR-002	Functional	The system shall split a customer order containing SKUs from more than one supplier partner into separate per-supplier sub-orders, each linked to the original order ID.	High	Pass: An order with items from 2 suppliers generates 2 supplier POs, each referencing the parent order ID. Fail: A multi-vendor order generates a single merged PO or loses the parent-order link.	Suppliers fulfil and ship independently, so orders must be correctly partitioned to avoid missed, duplicated, or cross-supplier shipments.
FR-003	Functional	The system shall update the corresponding sub-order with the tracking number and carrier name whenever a supplier partner reports shipment dispatch via their API or webhook.	Medium	Pass: Tracking number is attached to the correct sub-order within 5 minutes of the supplier's dispatch notification. Fail: A sub-order confirmed as shipped by the supplier has no tracking number after 24 hours.	Store owners and customers need near real-time shipment visibility without contacting the supplier directly.
FR-004	Functional	The system shall notify the E-Store Owner when a supplier partner's inventory API reports an out-of-stock or insufficient-quantity status for an ordered SKU, holding that line item for manual review.	Medium	Pass: A dashboard/email notification is raised within 1 minute of the stockout being detected. Fail: The affected order line fails silently with no notification to the E-Store Owner.	Prevents unresolved failed orders from reaching the customer unnoticed and keeps a human in the loop for exception handling.
FR-005	Functional	The system shall automatically retry a failed supplier API call (timeout or 5xx	Low	Pass: A simulated transient supplier API failure recovers automatically on retry	Improves resilience to brief supplier-side or network outages

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
		response) up to 3 times with exponential backoff before flagging the order for manual intervention.		without operator action. Fail: The order is abandoned after a single failed attempt.	without requiring constant manual monitoring.
NFR-001	Nonfunctional	Inbound webhook processing must acknowledge receipt with HTTP 200 within 200 ms, to prevent a provider webhook retry storm, and must authenticate the payload signature before processing.	High	Pass: Benchmark tests confirm p99 acknowledgment latency under 200 ms and reject unsigned/invalid payloads under simulated peak load.	Storefront platforms retry webhooks aggressively on slow or missing acknowledgments, which would otherwise cause duplicate order processing.
NFR-002	Nonfunctional	The system shall process webhook deliveries idempotently, so that receiving the same order-webhook payload more than once (e.g., a provider retry) never creates more than one supplier purchase order for that order.	High	Pass: Sending an identical webhook payload twice results in exactly one supplier PO across 100 simulated duplicate-delivery test runs. Fail: Any duplicate PO is created for a repeated payload.	Webhook providers commonly retry on timeouts or ambiguous acknowledgments; without idempotency this risks duplicate supplier POs and double-fulfilled orders.

Note: FR-001 and NFR-001 descriptions above follow the sample requirements given in the problem statement handout; FR-002 through FR-005 and NFR-002 are original requirements written to cover order-splitting, shipment tracking, stockout handling, retry resilience, and webhook idempotency.

2. USE-CASE DIAGRAM:



3. USE- CASE FLOW: PROCESS INCOMING ORDER:

Use Case ID	UC-02
Use Case Name	Process Incoming Order
Primary Actor	Customer (via storefront webhook); system acts on the E-Store Owner's behalf
Supporting Actors	Supplier Partner, E-Store Owner
Related Requirements	FR-001, FR-002, FR-004, NFR-001, NFR-002

Preconditions:

- The storefront platform (Shopify/WooCommerce) is registered and its webhook endpoint is authenticated with the system.
- At least one Supplier Partner is linked with a valid, reachable inventory API.
- The customer has completed checkout and payment on the storefront.

Postconditions (on success):

- One purchase order per supplier involved in the order has been generated and dispatched to the relevant Supplier Partner(s), including the customer's delivery address.
- The order's status is updated to "Processing" and each supplier PO reference is stored against the order.
- The storefront's webhook call received an HTTP 200 acknowledgment within 200 ms (NFR-001).

Main Success Scenario:

1. The storefront platform sends an order-created webhook to the system's ingestion endpoint.
2. The system authenticates the payload signature and immediately returns an HTTP 200 acknowledgment (NFR-001).
3. The system checks the order ID against previously processed webhooks to guarantee idempotent handling, then parses the payload to extract the order ID, SKUs, quantities, and customer delivery address (NFR-002).
4. For each supplier linked to the order's SKUs, the system queries that Supplier Partner's inventory API for stock availability [includes UC-03: Verify Supplier Inventory].
5. Every queried supplier confirms sufficient stock for its SKUs.
6. If the order's SKUs span more than one supplier, the system splits the order into separate per-supplier sub-orders, each referencing the parent order ID [includes UC-04: Split Multi-Vendor Order].
7. The system auto-generates and dispatches a purchase order to each relevant Supplier Partner, attaching the customer's delivery address.
8. The system updates the order status to "Processing" and stores the supplier PO reference numbers against the order.
9. Use case ends successfully.

Alternate Flow — 5a: Out-of-Stock SKU Detected:

- a. One or more Supplier Partners' inventory API report the requested SKU as out of stock or insufficient in quantity.
- b. The system does not generate a purchase order for the affected SKU and leaves the rest of the order flow unaffected.

- c. The system flags the order line item and notifies the E-Store Owner via the dashboard/email within 1 minute (FR-004).
- d. The E-Store Owner reviews the flagged line and either (i) cancels that line item and triggers a refund on the storefront platform, or (ii) places it on backorder pending supplier restock.
- e. The use case resumes at step 6 of the main success scenario for any remaining in-stock SKUs, then ends.

Exception Flow — Supplier API Unreachable:

- If a Supplier Partner's inventory API times out or returns a server error, the system retries up to 3 times with exponential backoff (FR-005) before flagging the order for manual intervention by the E-Store Owner.
-